

PENNYWISE

A PROJECT REPORT

on

PennyWise: AI-Powered Personal Finance & Behavioral Economics Platform

Submitted to

KIIT Deemed to be University

In Partial Fulfillment of the Requirement for the Award of

BACHELOR'S DEGREE IN

COMPUTER SCIENCE AND ENGINEERING

BY

[YOUR FULL NAME]

[YOUR ROLL NUMBER]

INDIVIDUAL PROJECT

School of Computer Engineering

KIIT Deemed to be University

Bhubaneswar, Odisha - 751024

[MONTH YEAR]

CERTIFICATE

This is to certify that the project entitled "PennyWise: AI-Powered Personal Finance & Behavioral Economics Platform" is submitted by [YOUR FULL NAME], [YOUR ROLL NUMBER], in partial fulfillment of the requirements for the award of the Bachelor's Degree in Computer Science and Engineering at KIIT Deemed to be University.

The project is an individual work developed by the student. No group contribution is applicable.

The project guide / supervisor information is intentionally left as a placeholder because the submitted materials do not specify a supervisor.

Role	Name / Signature	Date
Project Guide / Supervisor	[PROJECT GUIDE NAME — if required]	[DATE]
Student	[YOUR FULL NAME]	[DATE]

Acknowledgement

I would like to express my sincere gratitude to KIIT Deemed to be University and the School of Computer Engineering for providing the academic environment and facilities required for the development of this project.

I also acknowledge the technical documentation, open-source software libraries, and development tools that supported the implementation of PennyWise. The project was developed individually; therefore, no team contribution or external project supervisor is being claimed in this report.

Any remaining institutional, faculty, or supervisor acknowledgement can be added here if required by the department: [OPTIONAL ACKNOWLEDGEMENT PLACEHOLDER].

ABSTRACT

PennyWise is an AI-powered personal finance and behavioral economics platform designed to go beyond conventional expense tracking. The application combines financial record management with behavioral insights, predictive-style decision support, gamification, savings goals, subscription analysis, and interactive analytics. Its objective is to help users understand not only where their money goes, but also why particular spending patterns occur and how those patterns can affect future financial goals.

The current application is implemented as a full-stack web application using HTML5, CSS3 and Vanilla JavaScript on the frontend, Node.js with Express.js on the backend, JWT-based authentication, Chart.js for data visualization, and PDFKit for report generation. The database layer contains a relational schema covering users, settings, income, expenses, budgets, savings goals, subscriptions, achievements, challenges, notifications, reports and other behavioral-finance data. The project also includes a PostgreSQL production schema and a SQLite-to-PostgreSQL migration script for deployment to a managed database such as Neon.

The AI Behavior Suite includes rule-based decision support such as the 'Should I Buy It?' Advisor, a weighted Regret Predictor, Opportunity Cost Calculator, Future Savings Simulator, Hidden Expense Detector, Financial Time Machine, Weekly Spending Roast, and a Personalized Financial Coach. The dashboard also calculates financial health indicators from income, deductions, spending, impulse purchases, savings and mood-related expense information.

PennyWise further uses gamification through XP, levels, badges and daily savings missions, while Dream Savings Goals translate long-term financial targets into visible progress and level progression. The application is designed with a modern glassmorphism interface, responsive cards, interactive charts, dark/light theme support and a single-page navigation experience.

Keywords: Personal Finance, Behavioral Economics, AI Decision Support, Expense Tracking, Regret Prediction, Gamification, Savings Goals, Node.js, Express.js, JWT, PostgreSQL

Contents

The following structure follows the supplied project-report format. Page numbers can be refreshed after final editing in Microsoft Word.

Section	Title
1	Introduction
2	Basic Concepts
	Literature Review
3	Problem Statement and Requirement Specifications
3.1	Project Planning
3.2	Project Analysis (SRS)
3.3	System Design
3.3.1	Design Constraints
3.3.2	System Architecture / Block Diagram
4	Implementation
4.1	Methodology
4.2	Testing / Verification Plan
4.3	Result Analysis / Screenshots
4.4	Quality Assurance
5	Standard Adopted
5.1	Design Standards
5.2	Coding Standards
5.3	Testing Standards
6	Conclusion and Future Scope
6.1	Conclusion
6.2	Future Scope
	References

LIST OF FIGURES

Figure 3.1 High-Level System Architecture

Figure 3.2 Use Case Diagram

Figure 3.3 Entity-Relationship (ER) Diagram

Figure 3.4 Data Flow / Request-Response Flow

Figure 4.1 PennyWise Dashboard

Figure 4.2 Registration / Login Interface

Figure 4.3 Expense Management Interface

Figure 4.4 Analytics and Mood Analysis

Figure 4.5 AI Behavior Suite

Figure 4.6 Should I Buy It? Advisor Result

Figure 4.7 Regret Predictor Result

Figure 4.8 Dream Savings Goals

Figure 4.9 Subscription Killer

Figure 4.10 Money Game, XP and Badges

Figure 4.11 No-Spend Calendar

Figure 4.12 Monthly Report / PDF

Figure 4.13 Profile and Deduction Management

Figure 4.14 Future Savings / Opportunity Cost Tools

Chapter 1

Introduction

Personal finance applications traditionally focus on recording income, expenses and budgets. PennyWise extends this model by treating spending as a behavioral process. Instead of showing only transaction totals, it uses transaction history, mood information, impulse indicators, savings goals and subscription usage to generate actionable feedback.

The project was conceived as an AI-powered financial command center. The supplied project specification describes the platform as a personal finance and budgeting system that combines behavioral insights, gamification, predictive analytics and intelligent recommendations. The current codebase implements this direction through a browser-based single-page interface and a Node.js/Express REST API.

1.1 Background

Financial decisions are affected by factors such as affordability, emotional state, purchase frequency, recurring commitments and progress toward longer-term goals. A user may know how much was spent but still lack an immediate explanation of the behavioral consequences. PennyWise attempts to bridge this gap through contextual calculations and user-specific recommendations.

1.2 Motivation

The motivation behind PennyWise is to make financial management more understandable and actionable. The platform combines ordinary financial records with behavioral indicators so that users can receive explanations such as whether a purchase fits the available balance, how a purchase may delay a savings goal, whether subscriptions are being underused, and how small expenses accumulate.

1.3 Objectives

- Develop a unified personal-finance dashboard for income, deductions, expenses, budgets and savings.
- Provide user authentication and isolate user-specific financial records.
- Generate data-driven behavioral insights from transaction history and mood information.
- Provide decision-support tools for purchases, opportunity cost, future savings and regret risk.
- Encourage saving through Dream Goals, XP, levels, badges and daily missions.
- Provide subscription usage analysis and identify potential savings.
- Generate monthly financial reports in PDF format.
- Provide a modern, responsive and interactive user interface.
- Prepare the application for persistent cloud deployment using PostgreSQL and environment-based configuration.

1.4 Scope

The scope covers a web-based personal finance platform for individual users. It includes financial data entry, analytics, behavioral decision support, savings goals, subscriptions, gamification, reports, profile settings and supporting APIs. Deployment-specific details such as the final Vercel URL, Neon database URL, production verification results and final environment variables are intentionally left as placeholders until the deployment is completed.

1.5 Key Features

Area	Current Project Capability
Financial management	Income, expenses, deductions, budgets and monthly calculations
Behavioral intelligence	Purchase advisor, regret scoring, mood/impulse analysis, hidden expense detection
Savings	Dream Goals, savings progression, opportunity cost and future savings simulation
Subscriptions	Subscription records, usage marking, analytics and savings opportunities
Gamification	XP, levels, badges and daily savings missions
Analytics	Spending trends, category breakdowns, mood-related analysis and no-spend calendar
Reporting	Monthly report generation and PDF download
Security	JWT authentication, bcrypt password hashing and user_id based data filtering
Deployment	PostgreSQL schema, migration script and Vercel configuration included in project

Chapter 2

Basic Concepts

2.1 Personal Finance Management

Personal finance management involves tracking income, recurring deductions, spending, savings and financial goals. PennyWise represents these entities relationally and calculates summaries such as total income, deductions, take-home income, expenses and savings.

2.2 Behavioral Economics in Personal Finance

Behavioral economics examines how human decisions can differ from purely rational financial choices. PennyWise applies this concept through mood tags, impulse indicators, purchase satisfaction, regret risk, spending patterns and contextual recommendations. These mechanisms are implemented as application logic rather than as a claim of clinical or psychological diagnosis.

2.3 Rule-Based AI and Decision Support

The current AI Behavior Suite is primarily rule-based and data-driven. It combines thresholds, weighted scores, historical database queries and financial calculations. This makes the behavior transparent and reproducible: the user can be shown the components contributing to a recommendation.

2.4 Regret Prediction Mechanism

The Regret Predictor calculates a 0–100 weighted score using six components: income risk (25%), category risk (20%), mood risk (15%), impulse risk (15%), Dream Goal risk (15%) and purchase-history risk (10%). The resulting score is mapped to Low, Moderate, High or Very High risk. The system also produces reasons, a goal-delay estimate and a recommendation. Feedback can be stored through the purchase-feedback endpoint for future tuning.

2.5 Authentication and Authorization

PennyWise uses JSON Web Tokens (JWT) to authenticate protected API requests. The authentication middleware extracts the token and associates requests with a user ID. Protected database operations include the authenticated user ID in queries, supporting user-specific data isolation.

2.6 Relational Database

The project contains both a local SQLite schema and a PostgreSQL production schema. The PostgreSQL schema defines 21 application tables covering users, settings, deductions, login history, income, categories, expenses, budgets, savings goals, shared goals, subscriptions, subscription history, savings insights, purchase feedback, price watch, achievements, user achievements, challenges, receipts, notifications and reports. A compatibility view named `dream_goals` is also defined over `savings_goals`.

2.7 REST API Architecture

The backend exposes REST-style endpoints grouped by responsibility: authentication, dashboard and goals, expenses, AI behavior, subscriptions and gamification. The frontend communicates with these APIs asynchronously using fetch-based requests.

2.8 Data Visualization

Chart.js is used for interactive financial visualization. The dashboard includes spending trends and category-oriented views, while analytics sections provide additional financial and mood-related visualizations.

2.9 PDF Report Generation

PDFKit is included in the backend and is used by the reporting module to generate monthly financial summaries. The report functionality records generated reports in the reports table and provides a download endpoint.

2.10 Gamification

Gamification converts financial progress into XP, levels, badges and missions. Dream Goals also calculate a progression level and unlocked title based on the percentage of the target amount saved.

2.11 Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	Node.js, Express.js
Database	SQLite for local development; PostgreSQL schema for production
Authentication	JWT + bcryptjs
Visualization	Chart.js
Reports	PDFKit
Testing	Jest, Supertest; Playwright test file included
Deployment configuration	Vercel configuration; Neon PostgreSQL migration support

2.12 Project Modules

- 1. Authentication (Register, Login, Forgot Password)
- 2. Dashboard / Financial Command Center
- 3. Income Management
- 4. Expense Management
- 5. Budget Planner
- 6. Analytics & Charts
- 7. No-Spend Calendar
- 8. Monthly Reports
- 9. Profile & Settings
- 10. Should I Buy It? Advisor
- 11. Regret Predictor

- 12. Subscription Killer
- 13. Daily Savings Missions
- 14. XP, Levels & Badges
- 15. Personalized Financial Coach
- 16. Impulse Purchase Countdown Lock
- 17. Dream Savings Goals
- 18. Weekly Spending Roast
- 19. Opportunity Cost Calculator
- 20. Future Savings Simulator
- 21. Hidden Expense Detector / Financial Time Machine

The original project specification also lists OCR receipt scanning, price memory/price-drop alerts, carbon footprint, location heatmap and accountability/shared goals. The current codebase contains database/UI traces for some of these areas, but final implementation status should be verified before the final submission. [VERIFY FEATURE STATUS BEFORE FINAL REPORT SUBMISSION].

Literature Review

The project specification positions PennyWise/FinPilot AI within the intersection of personal finance software, behavioral economics and intelligent recommendation systems. Conventional finance applications emphasize transaction recording and visual summaries. PennyWise attempts to extend that model by connecting transaction data to behavioral context and future goals.

From a software-engineering perspective, the project follows a modular REST API architecture, separating authentication, financial records, AI behavior, subscriptions and gamification. This separation makes it possible to evolve individual modules without redesigning the complete interface.

From a behavioral perspective, the system uses observable application data such as purchase amount, category, mood and impulse markers. The Regret Predictor is intentionally transparent: its weighted components are exposed in the response rather than hiding the calculation behind an opaque model.

From a user-experience perspective, the project uses a glassmorphism visual system, interactive cards, progress indicators, charts and game-like rewards to make financial information more engaging.

For the final academic submission, add the specific research papers, books, standards and official technical documentation consulted during development here: [LITERATURE REFERENCES TO BE ADDED].

Chapter 3

Problem Statement and Requirement Specifications

3.1 Problem Statement

Many personal finance tools provide transaction records and charts but do not connect individual purchases to behavioral patterns, savings goals and decision-making context. A user may therefore see an expense total without understanding whether the purchase was impulsive, how it affects a target, whether recurring services are being underused, or what future savings could result from reducing a category.

PennyWise addresses this problem by integrating financial tracking with behavioral decision support, gamification and goal-oriented feedback in a single web application.

3.2 Project Planning

Phase 1 – Requirement Identification: Identify financial-management, behavioral and gamification requirements.

Phase 2 – Interface and Data Design: Define the SPA structure, database entities, user flows and dashboard components.

Phase 3 – Backend/API Development: Implement Express routes for authentication, dashboard, expenses, AI, subscriptions and gamification.

Phase 4 – Behavioral Intelligence: Implement rule-based financial calculations, regret scoring, recommendations and simulations.

Phase 5 – Frontend Integration: Connect UI components to REST endpoints and render returned data dynamically.

Phase 6 – Testing: Use unit, integration and browser-level tests to validate major workflows.

Phase 7 – Production Preparation: Prepare PostgreSQL schema, migration tooling, environment variables and Vercel deployment configuration.

3.3 Project Analysis (SRS)

3.3.1 Functional Requirements

- User registration, login, JWT authentication, profile access and account deletion.
- Management of income, deductions, expenses and budgets.
- Creation, update and deletion of Dream Savings Goals.
- Addition of savings to goals with progress-level calculation.
- Subscription management, usage marking and subscription analytics.
- Dashboard summaries including income, deductions, expenses, savings and financial health.
- AI behavior tools including purchase advice, regret prediction, opportunity cost and future savings simulation.

- Gamification through XP, levels, badges and daily missions.
- Monthly report generation and PDF download.
- User-specific access control for protected records.

3.3.2 Non-Functional Requirements

- Security: passwords must not be stored in plaintext and protected routes must require authentication.
- Maintainability: routes and database responsibilities should remain modular.
- Responsiveness: the interface should adapt to common desktop and mobile viewport sizes.
- Reliability: user data should persist in the production database rather than browser-only or local-server storage.
- Performance: common dashboard and CRUD operations should return promptly under normal usage.
- Scalability: the production database and serverless-compatible connection approach should support future growth.
- Configuration safety: credentials and deployment secrets must be supplied through environment variables.

3.3.3 Design Constraints

Constraint	Description
Runtime	Node.js + Express.js application
Frontend	Vanilla JavaScript SPA with HTML/CSS
Authentication	JWT-based protected API
Local database	SQLite remains useful for local development/testing
Production database	PostgreSQL / Neon
Deployment	Vercel-compatible backend configuration
External services	Any external AI/API keys must be supplied through environment variables
Browser	Modern JavaScript-capable browser required

3.4 System Design

The application follows a layered structure. The browser provides the presentation layer. JavaScript functions call Express REST endpoints. Route modules perform authentication checks and business logic, while the database wrapper abstracts SQLite/PostgreSQL access. The PostgreSQL schema is intended for persistent production storage.

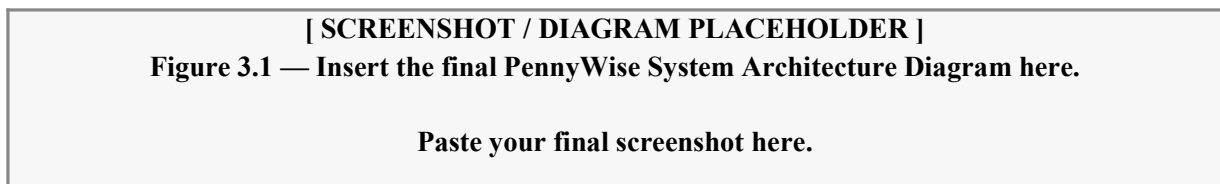


Figure 3.1: High-Level System Architecture

3.4.1 Use Case Model

The primary actor is the authenticated user. Major use cases include registration/login, recording income and expenses, viewing analytics, creating Dream Goals, using AI decision-support tools, managing subscriptions, completing savings missions and downloading reports.

[SCREENSHOT / DIAGRAM PLACEHOLDER]
Figure 3.2 — Insert the final Use Case Diagram here.

Paste your final screenshot here.

Figure 3.2: Use Case Diagram

3.4.2 Entity-Relationship Model

The relational model is centered on the users entity. User-specific tables reference users through user_id, while dependent records such as expenses, budgets, savings goals, subscriptions, challenges and notifications can be removed through cascading relationships. This supports account-level data isolation and cleanup.

[SCREENSHOT / DIAGRAM PLACEHOLDER]
Figure 3.3 — Insert the final ER Diagram based on db/schema.postgres.sql here.

Paste your final screenshot here.

Figure 3.3: Entity-Relationship Diagram

3.4.3 Data Flow

A typical request follows this flow: user action → frontend fetch request → Express route → JWT authentication → database query/business logic → JSON response → frontend rendering. For AI tools, the route additionally computes behavioral metrics before returning the score, explanation and recommendation.

[SCREENSHOT / DIAGRAM PLACEHOLDER]
Figure 3.4 — Insert a request/response data-flow diagram here.

Paste your final screenshot here.

Figure 3.4: Data Flow / Request-Response Flow

Chapter 4

Implementation

4.1 Methodology

PennyWise was implemented incrementally around a modular Express backend and a single-page Vanilla JavaScript frontend. The codebase separates database access from route-level business logic and uses protected endpoints for user-specific operations.

4.1.1 Authentication Module

The authentication module supports registration, login, profile access/update, deductions, forgot-password flow and account deletion. Passwords are processed using bcryptjs and successful authentication returns a JWT used by protected API calls.

4.1.2 Expense and Income Management

Expenses contain fields such as title, amount, category, location, mood, impulse flag, satisfaction score, carbon score and date. Income records store title, amount, category, frequency and date. CRUD operations are scoped to the authenticated user.

4.1.3 Dashboard and Financial Health

The dashboard aggregates income, deductions and expenses, compares current and previous month spending, ranks expense categories and calculates a financial health score. The score uses savings rate, impulse frequency, expense ratio, goal progress and mood stability as contributing factors.

4.1.4 AI Behavior Suite

- Should I Buy It? Advisor evaluates available balance and the possible impact of a purchase on a savings goal.
- Regret Predictor computes six weighted risk components and returns a 0–100 score, risk level, reasons, goal delay and recommendation.
- Opportunity Cost Calculator converts a purchase cost into simple everyday equivalents.
- Future Savings Simulator estimates monthly, one-year and five-year savings from reducing a spending category.
- Hidden Expense Detector identifies small-value transactions and estimates their cumulative impact.
- Weekly Spending Roast generates a humorous observation from the user's spending categories.
- Financial Time Machine and Coach provide additional decision-support calculations/messages.

4.1.5 Dream Savings Goals

Dream Goals are stored in `savings_goals`. A goal has a title, target amount, current amount, target date, category, destination, theme, emoji and progression fields. Goal progress is mapped

to levels such as Penny Beginner, Budget Explorer, Treasure Hunter, Savings Knight, Wealth Wizard and Financial Legend.

4.1.6 Subscription Killer

The subscription module stores recurring services and their monthly costs. It supports creating, updating, deleting and analyzing subscriptions, including usage-related savings opportunities.

4.1.7 Gamification

Gamification stores achievements and user achievements and exposes daily challenges. Completing a challenge updates XP and the user's level. Shared goals are also returned as part of the gamification data.

4.1.8 Reporting

The reporting module creates monthly summary records and generates PDF output using PDFKit. Final production verification of the PDF download URL and storage behavior should be completed after deployment.

4.1.9 Database and Production Migration

The project includes `db/schema.sqlite.sql` for local storage and `db/schema.postgres.sql` for PostgreSQL. The script `scripts/migrate_to_postgres.js` reads the existing SQLite database and transfers records into PostgreSQL, including boolean conversion, conflict handling and sequence reset logic. The migration is intended to preserve existing user and financial data while moving persistent production storage to a managed PostgreSQL service.

4.1.10 Deployment Architecture

The intended production architecture is GitHub → Vercel → Neon PostgreSQL. The Vercel configuration maps the Express server entry point to a Vercel Node function. The database URL is provided through `DATABASE_URL` rather than hardcoded credentials. The final deployment URL and production verification results are placeholders until the deployment is complete.

Deployment Item	Final Value
GitHub Repository	[INSERT GITHUB REPOSITORY URL]
Vercel Project	[INSERT VERCEL PROJECT NAME / URL]
Production URL	[INSERT FINAL PENNYWISE URL]
Neon Project	PennyWise Project / [VERIFY FINAL NAME]
Database	PostgreSQL / Neon
DATABASE_URL	[DO NOT PLACE THE SECRET VALUE IN THE REPORT]
Deployment Date	[INSERT DATE]

4.2 Testing / Verification Plan

The project contains automated tests using Jest and Supertest for authentication, expenses, AI behavior, budgets and integration flows. A Playwright browser test file is also included for frontend navigation and UI checks. The test suite should be rerun against the final production configuration before the report is submitted.

Test Area	Verification
Registration / Login	Create account, authenticate and verify protected access
Expense CRUD	Add, list, edit, satisfaction update and delete expense
Budget	Create, update, view and delete budget
AI Advisor	Verify recommendation and explanation
Regret Predictor	Verify numeric score, risk, reasons and recommendation
Purchase Feedback	Verify feedback is stored
Subscription Killer	Verify subscription list and savings opportunities
Gamification	Verify XP, badges, missions and shared goals
Reports	Generate monthly report and verify PDF download
Account deletion	Verify user and dependent data are removed
Multi-user isolation	Verify one user's token cannot retrieve another user's records
Production persistence	Verify data survives logout, redeployment and laptop shutdown

Final test evidence: [INSERT TEST RUN OUTPUT / SCREENSHOT / PASS-FAIL SUMMARY HERE].

4.3 Result Analysis / Screenshots

The following placeholders are intentionally left empty so the final report can contain screenshots from the deployed PennyWise application rather than development or temporary states.

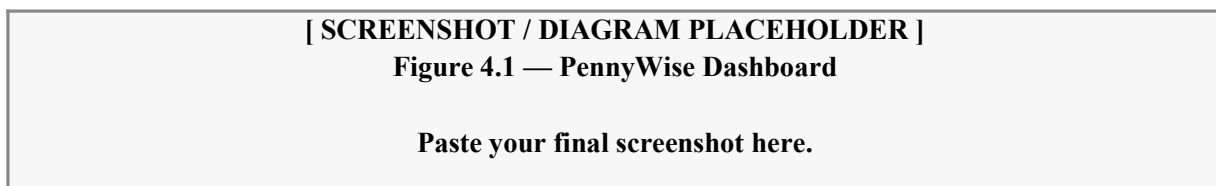


Figure 4.1: PennyWise Dashboard

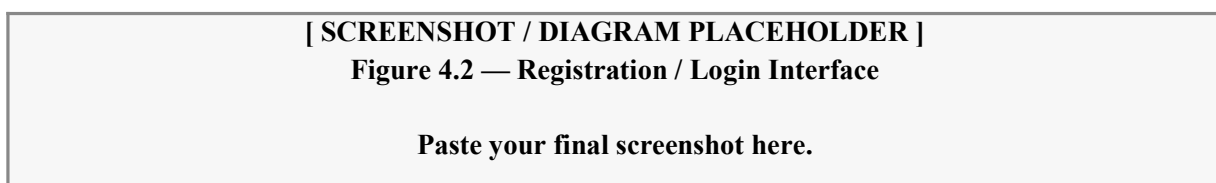


Figure 4.2: Registration / Login Interface

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.3 — Expense Management Interface

Paste your final screenshot here.

Figure 4.3: Expense Management Interface

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.4 — Analytics and Mood Analysis

Paste your final screenshot here.

Figure 4.4: Analytics and Mood Analysis

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.5 — AI Behavior Suite

Paste your final screenshot here.

Figure 4.5: AI Behavior Suite

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.6 — Should I Buy It? Advisor Result

Paste your final screenshot here.

Figure 4.6: Should I Buy It? Advisor Result

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.7 — Regret Predictor Result

Paste your final screenshot here.

Figure 4.7: Regret Predictor Result

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.8 — Dream Savings Goals

Paste your final screenshot here.

Figure 4.8: Dream Savings Goals

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.9 — Subscription Killer

Paste your final screenshot here.

Figure 4.9: Subscription Killer

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.10 — Money Game, XP and Badges

Paste your final screenshot here.

Figure 4.10: Money Game, XP and Badges

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.11 — No-Spend Calendar

Paste your final screenshot here.

Figure 4.11: No-Spend Calendar

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.12 — Monthly Report / PDF

Paste your final screenshot here.

Figure 4.12: Monthly Report / PDF

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.13 — Profile and Deduction Management

Paste your final screenshot here.

Figure 4.13: Profile and Deduction Management

[SCREENSHOT / DIAGRAM PLACEHOLDER]

Figure 4.14 — Future Savings / Opportunity Cost Tools



Paste your final screenshot here.

Figure 4.14: Future Savings / Opportunity Cost Tools

4.4 Quality Assurance

Quality assurance in PennyWise is supported through modular route separation, authentication middleware, input handling, database constraints, automated tests and deployment configuration. The codebase also contains a migration utility and a PostgreSQL schema to reduce the risk of losing local financial data during production migration.

Before final submission, complete the following QA checks: [FINAL QA CHECKLIST TO BE COMPLETED AFTER DEPLOYMENT].

Chapter 5

Standard Adopted

5.1 Design Standards

- Modular separation of frontend, route handlers and database access.
- Consistent REST endpoint naming under /api.
- Responsive card-based interface with reusable CSS classes.
- Consistent authentication middleware for protected resources.
- Relational foreign keys using user_id for user-owned data.
- Use of environment variables for secrets and deployment configuration.
- Use of clear status messages and JSON responses for API operations.

5.2 Coding Standards

- JavaScript functions use descriptive names such as loadDashboardData, runRegretPredictor and addDreamGoal.
- Database parameters are passed separately from SQL strings through the query wrapper.
- Route modules are grouped by functional responsibility.
- Error handling uses try/catch blocks in asynchronous route handlers.
- Configuration secrets are excluded through .gitignore and represented through .env.example.
- Comments are used to identify major API modules and business-logic sections.

5.3 Testing Standards

- Authentication tests verify protected access and account cleanup.
- CRUD tests verify that records can be created, retrieved, modified and deleted.
- AI tests verify response structure and core outputs.
- Integration tests verify multi-step workflows across modules.
- Browser tests verify important frontend navigation and interaction paths.
- Final deployment testing should include persistent database behavior and user isolation.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

PennyWise brings together personal finance tracking, behavioral decision support and gamification in a single full-stack web application. The current project contains a broad relational data model, JWT-protected APIs, a dynamic browser interface, AI-style behavioral calculations, Dream Savings Goals, subscription analysis, gamification and PDF reporting.

A key strength of the project is that several behavioral features are transparent and explainable. For example, the Regret Predictor exposes its contributing risk components, while the financial health score is calculated from identifiable financial and behavioral indicators. This makes the system easier to understand than a completely opaque recommendation mechanism.

The production architecture is designed to move persistent data from local SQLite storage to PostgreSQL/Neon and deploy the application through Vercel. The final deployment status, production URL, migration verification and end-to-end acceptance results should be inserted after those steps are completed.

6.2 Future Scope

- Complete and verify the Vercel + Neon production deployment and automated deployment pipeline.
- Introduce a dedicated serverless-safe database access pattern and optimize connection pooling for high concurrency.
- Replace selected rule-based components with trained machine-learning models after collecting sufficient user-consented data.
- Add stronger personalized learning to the Regret Predictor using historical purchase feedback.
- Expand receipt OCR and price-watch functionality if these modules are retained in the final product scope.
- Add more advanced spending forecasts and anomaly detection.
- Introduce mobile/PWA support.
- Add stronger privacy controls, data export and consent management.
- Add more detailed admin analytics if an administrative role is required.
- Improve test coverage for production database migration and multi-user isolation.

References

- [1] Node.js Documentation — [INSERT OFFICIAL URL AND ACCESS DATE].
- [2] Express.js Documentation — [INSERT OFFICIAL URL AND ACCESS DATE].
- [3] PostgreSQL Documentation — [INSERT OFFICIAL URL AND ACCESS DATE].
- [4] Neon PostgreSQL Documentation — [INSERT OFFICIAL URL AND ACCESS DATE].
- [5] Vercel Documentation — [INSERT OFFICIAL URL AND ACCESS DATE].
- [6] JSON Web Token (JWT), RFC 7519 — [INSERT/VERIFY CITATION DETAILS].
- [7] Chart.js Documentation — [INSERT OFFICIAL URL AND ACCESS DATE].
- [8] PDFKit Documentation — [INSERT OFFICIAL URL AND ACCESS DATE].
- [9] bcrypt / bcryptjs documentation — [INSERT OFFICIAL URL AND ACCESS DATE].
- [10] Additional academic papers or sources used for behavioral economics / personal finance — [ADD SOURCES].

Note: The reference list is intentionally left partly as placeholders because the supplied project files do not contain a finalized academic reference list. Replace these entries with the exact sources actually consulted.

Final Submission Checklist

This checklist is included for final editing and should be removed if the department does not require it.

- Replace [YOUR FULL NAME] and [YOUR ROLL NUMBER].
- Add supervisor/guide information only if the department requires it; otherwise keep the individual-project wording.
- Insert final system architecture, use case, ER and data-flow diagrams.
- Insert screenshots from the final deployed PennyWise application in the marked spaces.
- Insert the final GitHub repository and Vercel URL.
- Confirm Neon PostgreSQL migration and record final deployment verification results.
- Run the complete automated test suite against the final code.
- Perform manual tests for authentication, expenses, Dream Goals, subscriptions, reports, AI features and user isolation.
- Update the Contents and List of Figures page numbers in Word.
- Add the exact references actually used.
- Do not add a group contribution section or plagiarism report, as requested.